TERMWISE

Technical Design Document

One college record, from admission to graduation.

Repository	https://github.com/moneytosms/termwise
Live app	https://termwise-s1vj.onrender.com
Project site	https://moneytosms.github.io/termwise/
Date	November 2026

Contents

At a glance 3
How to read this 3
1. What the slice proves 4
2. Architecture 4
3. Data model 6
4. Main flows 7
5. How the AI part works 14
6. UI and design decisions 15
7. Testing and security 16
8. What was cut, and what comes next 17
9. Assumptions 18

At a glance

What it is. Termwise is the first phase of a college ERP (a system that runs a college's daily records) for Company A's trust, which runs several campuses.

Who it serves. Faculty, office staff (admin, admissions, exams, accounts, HR), heads of department, students, parents and the trust secretary. Each sees only what their role allows.

What works today:

- Attendance taken on a phone with no network, synced later, with conflicts sent to the office.
- Internal marks in components, a moderation lock, an upload CSV and a printable mark sheet with a verification page.
- Fee dues calculated from instalments, payments, scholarships, receipts and Tally export.
- An at-risk flag decided by fixed rules, and a plain-English question assistant (Termy) that can only read filtered views.
- Student and parent portal, trust dashboard, reports, AICTE and NAAC packs, admissions, staff leave, promotion and graduation.

What it is not. A demo for the assignment, not a production system. Messages are logged, not sent, and payments are test mode (section 8).

How to read this

- **In 5 minutes:** the At a glance block, then section 1 (what the brief needs), section 2 (architecture), 4.1 (offline attendance), 5 (the AI part) and 8 (what was cut).
- **Deep detail:** sections 3, 4.2 to 4.16 and 7 are reference. Dip in when a screen or rule raises a question.
- **Assumptions:** marked **[A]** in the text and all listed in section 9. Each is a choice the brief does not settle.
- **Every section opens with its takeaway** in bold, then the detail.

Terms used below:

Term	Meaning
Server action	A server function a page calls to change data; it checks role and campus first
Service worker	A script in the browser that serves pages when offline
Outbox	The phone's queue of saved attendance sheets waiting to sync (kept in IndexedDB, the browser's database)
Moderation lock	After review, a subject's marks can no longer be edited without a reason
HMAC hash	A keyed fingerprint that shows a receipt or mark sheet was not altered
DPDP	India's Digital Personal Data Protection Act; it requires parental consent for minors
AICTE, NAAC	Indian technical-education regulator and accreditation body, whose returns the Reports page prepares
Tally	Accounting software that imports the receipt vouchers
ADR	Architecture decision record, in `docs/adr/`
IST	Indian Standard Time
Termy	The in-app assistant that answers plain-English questions

1. What the slice proves

> **Takeaway: the design starts from attendance on a phone with weak Wi-Fi, and one record per student feeds marks, dues, risk and messages with no re-entry.**

Brief success measure	How the slice serves it
Attendance entered the same day for 95% of classes	Home opens on today's classes; marking a class is one tap per absentee and works with no network
University mark uploads with no re-typing	Marks are entered once per component, locked after moderation, exported in an upload CSV
Fee reconciliation from 10 days to 1	Dues are calculated from instalments and payments, never typed; receipts export as Tally vouchers

Shipped beyond the slice:

- **Trust dashboard.** A read-only view for the trust secretary across all campuses (section 4.6).
- **Term mark sheet.** A printable sheet for a student, with a verification hash, a QR code and a public verify page. Only locked subjects count (section 4.7).
- **Standard reports.** Strength, attendance shortage, marks and fees, with month and programme filters, CSV export and a monthly snapshot (section 4.8).
- **Odia parent messages.** Each guardian gets messages in their own language (section 4.5).
- **Precomputed results.** Each locked subject writes one result row per student, so the portal reads a row instead of recomputing marks (section 4.9).
- **Applicants and document checklist.** Applicants are imported from a CSV, each gets a checklist of the documents their programme requires, and the office tracks each document (section 4.10).
- **Confirm admission.** One step turns a complete applicant into a student, with a roll number, fee instalments and a guardian login (section 4.11).
- **Staff leave.** Faculty apply for casual, medical or on-duty leave and see their yearly balances. The office admin of the same campus approves or rejects it (section 4.12).
- **Multi-campus logins.** One login can hold a staff row on each campus. A sidebar switcher picks the active campus, and offline sheets sync under the row they were marked with (section 4.15).
- **Office roles and head of department.** Admissions, exams, accounts and HR each get the modules of their office, and a head of department sees the own department and decides its faculty leave (section 4.15).
- **Fee structure and scholarships.** Each programme can set a semester fee per category, and scholarship schemes are awarded per student per semester (section 4.13).
- **Promotion and graduation.** A batch moves to its next semester, or graduates into alumni, from College setup (section 4.14).
- **Accreditation packs.** AICTE and NAAC returns in Reports, with the sanctioned intake, posts and staff details they need (section 4.16).

2. Architecture

> **Takeaway: one Next.js app with one Postgres database, so there is one thing to deploy and no API contract to maintain.**

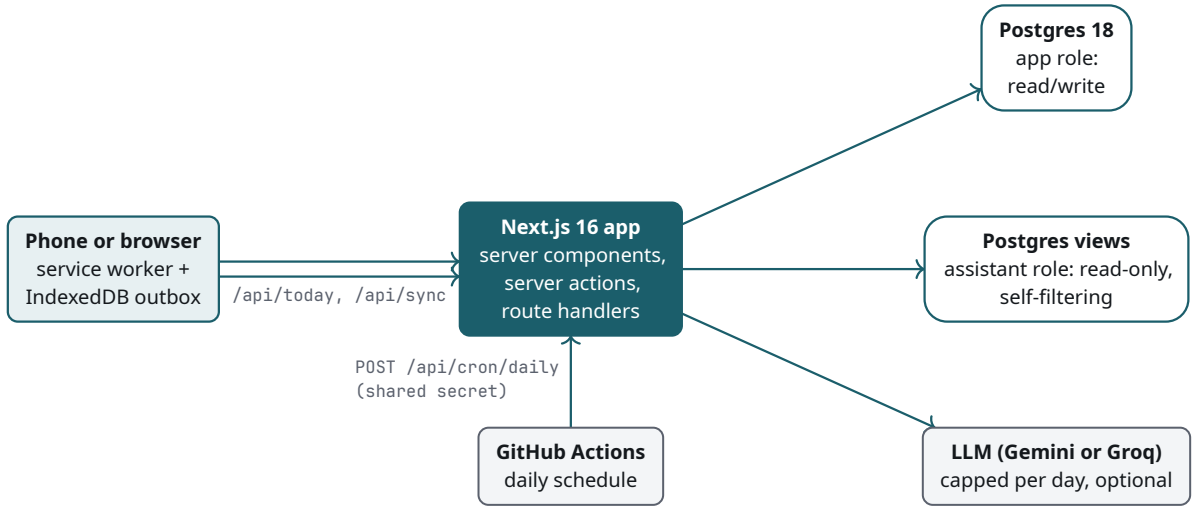


Figure 1. Architecture: one Next.js app, one Postgres database, two database roles.

One deployable. A single Next.js app with its database, in one Docker image. Server components read the database directly; mutations are server actions that check the viewer's role and campus, validate with zod and write an audit row in the same transaction. For a 6 to 8 screen slice built by one person in a week, a separate API service would add a contract to maintain and nothing the client can see. Route handlers exist only where a non-page client needs them: the offline sync endpoint, CSV, Tally and report downloads, the precomputed results API and the daily job.

Why these parts

- **Next.js and React** for one language across screens and server, and server components that keep student data on the server unless a screen needs it.
- **Postgres** because the domain is relational (student, batch, subject, session, entry) and because two database features carry the AI guardrails: views and roles.
- **Drizzle** for typed queries that stay close to SQL, so the attendance and dues arithmetic is readable.
- **Better Auth** with email and password. Campus roles live on `staff` rows linked to the user, one row per campus, so access checks are one join, not an organisation plugin.
- **Dexie (IndexedDB) and a hand-written service worker** for offline. A plugin-generated worker could not reliably serve the app's dynamic pages offline on Next 16, so the worker is about 70 lines that precache exactly three screens and their assets.
- **Vercel AI SDK** with a structured output schema, so the LLM returns a plan object, never free text that is executed.

Access control. Every query on a screen goes through `getViewer()`, which resolves the signed-in user to one of their staff rows: the active campus's row, chosen by the sidebar switcher and validated against the user's own rows (section 4.15). The row gives a role and a campus. Each office role sees only the modules its role holds (`CAPS` in `src/domain/roles.ts`, section 4.15). Faculty see the subjects they teach and the students they mentor. A head of department is limited to their department. Anyone visiting a screen outside their modules is sent to Home. The assistant, Termy, runs as a separate database role that can only read views filtered by campus, role, staff id and department.

3. Data model

> **Takeaway: a student exists once, and everything hangs off that row; money and percentages are calculated, never typed in.**

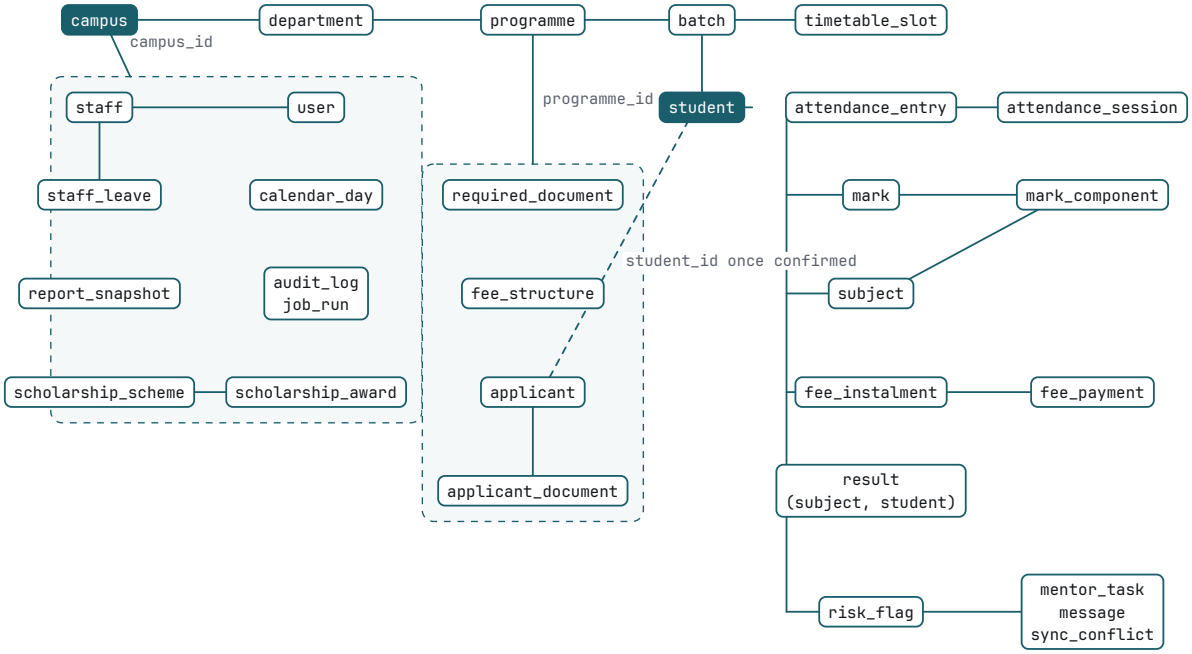


Figure 2. Data model: a student exists once and everything hangs off that row.

Choices that matter:

- **A student exists once.** Roll number is unique across the trust. Attendance, marks, dues, flags and messages all hang off `student.id`.
- **Attendance session, not attendance sheet.** One row per batch, subject, date and period, with a client-generated UUID so the phone can create it offline. A lab is one session covering several periods (`period_count`); percentages are weighted by periods. Late counts as attended **[A]**.
- **Dues are calculated, never stored.** Due = instalment amount − scholarship − payments. Overdue = due on instalments past their due date. No balance column can drift from the payments.
- **Marks scheme by section.** Each subject is theory or lab and has a maximum per section. Faculty define the internal components, which must total the internal maximum of 40 (the demo uses Internal test 20, Assignment 10, Quiz 5, Attendance 5 **[A]**). Mid-term exam (20) and end-term exam (40) are one component each; a lab adds lab exam (15), viva (10) and continuous evaluation (5) **[A]**. Internal marks are the internal section alone. The final score is out of 100: theory (internal, mid, end) scaled by the theory scale (100 for theory, 70 for labs), plus the lab sections. See `adr/0001-marks-scheme-internal-40.md`. A subject's marks move draft → submitted → locked. A locked subject rejects edits at the server; reopening needs a reason, and every change is in the audit log.
- **Appeals.** A student (not a parent) appeals one mark or one absent or late entry, with a reason of 10 to 500 characters. One open appeal per item. An accepted appeal can change a locked mark; it is audited with the appeal id.
- **College profile.** Legal name, tagline, affiliation, registration number, GSTIN, address, contact, logo (data URL, 200 KB at most) and receipt footer sit on the campus row. Receipts read them.

- **Rules are settings, not AI.** Attendance cutoff (75% default per programme), risk thresholds (attendance drop of 10 points in 4 weeks, internal marks below 50% of the maximum of the internal components entered, at least 10 sessions held **[A]**) live on the campus and programme rows and are edited in College setup.
- **Audit log** records actor, action, entity, old and new value and reason, written in the same transaction as the change.
- **Staff roles.** A staff row has one of seven roles: `faculty`, `office_admin`, `admissions`, `exams`, `accounts`, `hr` or `hod`. Each role's capabilities are the `CAPS` table in `src/domain/roles.ts` (section 4.15). A staff row has a campus. `department_id` is nullable and is set for `faculty` and `hod` rows. The `trust` role belongs to the trust secretary, has no campus of its own and is read-only everywhere (section 4.6). Only the trust area accepts it; every campus screen and action rejects it.
- **One login, several campuses.** A user can hold one staff row per campus, which is the seed convention. The `staff` table has no unique index, so nothing in the schema stops a second row for the same user and campus. The app assumes one row per campus **[A]**. The same person can be faculty at one campus and an admissions officer at another (section 4.15).
- **Results are precomputed.** `result` holds one row per subject and student, with the theory total (internal, mid-term and end-term, raw out of 100 **[A]**), the scaled theory, the lab total and the final score. It is written when a subject locks, deleted when it is reopened and recomputed when an accepted mark appeal changes a locked mark. Its `internal` column holds that raw theory total, not the internal section alone; the name is misleading and a later migration should rename it.
- **Report snapshots.** `report_snapshot` keeps one copy per campus, month and report (strength, shortage, marks, fees). A unique index on those three columns makes the monthly write idempotent (section 4.8).
- **Applicants stay apart until admission.** `applicant` holds the person, their category, guardian details, parent consent date and a status: `applied`, `confirmed` or `rejected` (with a reason). Only a confirmed applicant has a student row, linked by `applicant.student_id`, which is unique so one applicant becomes one student.
- **Checklist is stored per item.** `required_document` lists the documents a programme needs, with an optional list of categories (null means all). `applicant_document` holds one status per applicant and document: `pending`, `received` or `verified`. An applicable document with no row counts as pending, so the checklist is never out of step with the list.
- **Fees per programme and category.** `programme.semester_fee_paise` holds the default semester fee. `fee_structure` holds one semester fee per programme and category (unique on both), and a category with no row uses the default (section 4.13). Admission splits the fee into two instalments (section 4.11). Scholarship schemes and their awards are stored as `scholarship_scheme` and `scholarship_award` (section 4.13).
- **Leave is stored as requests.** `staff_leave` holds the kind (`casual`, `medical`, `on_duty`), the from and to dates, the reason, the status (`pending`, `approved`, `rejected`, `withdrawn`), the office note and the decider. Days used are counted from the dates, never stored, so balances cannot drift.

4. Main flows

> **Takeaway: the offline attendance flow (4.1) is the hard part and the core of the build. 4.2 to 4.4 are the rest of the brief; 4.5 to 4.16 are extras.**

4.1 Offline attendance with sync (the hard part)

1. When a teacher opens the app with any connection, `/api/today` returns today's classes with rosters (roll numbers and names only) and the phone stores them in IndexedDB. The service worker caches Home, Classes and Mark attendance.
2. In class, with no network, the teacher opens the class. Everyone starts present; one tap marks absent, a second tap late, a third tap present again. A search box jumps to a roll number. Save writes the sheet to an outbox in IndexedDB and returns to Home, where the class shows as queued.
3. The outbox flushes when the app opens, when the network returns and through Background Sync on Android. The server answers each item separately: saved, stale (an older edit), conflict or rejected, so one bad item never blocks the rest.
4. **Conflicts.** If another teacher already saved the same class, the incoming sheet goes to a conflict queue for the office admin instead of overwriting it. Last-edit-wins only applies to the same teacher's own edits. Edit times are corrected for a wrong phone clock using the gap between the phone's and the server's time at send.
5. **Shared phones.** Each queued sheet carries the id of the teacher who marked it. If a different teacher is signed in when it sends, the server rejects it and the sheet stays on the phone until the right teacher signs in.
6. Signing out clears everything about students from the phone.

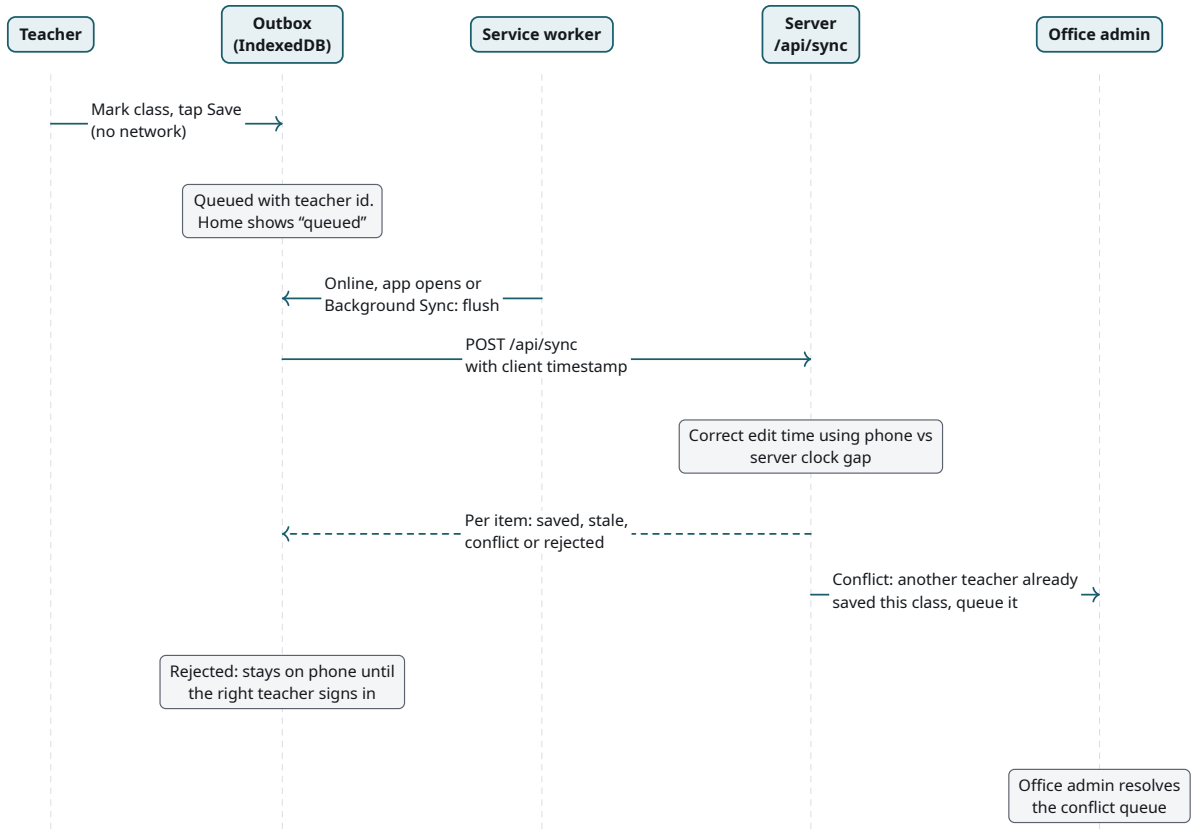


Figure 3. Offline attendance: from a tap with no network to a per-item outcome on the server.

4.2 Marks and moderation

Faculty enter marks per component on one sheet per subject, with each cell checked against the component's maximum, then save as draft and submit. The office admin locks a submitted subject; the lock is enforced in the server action, not just hidden in the UI. Reopen needs a typed reason. The locked sheet

exports as CSV in an assumed university upload layout: `RollNo, Name, SubjectCode`, one column per internal component, `InternalTotal` (internal section only) **[A]**.

4.3 Fees

Instalments per student (the seed uses two per year with some scholarships) produce dues and overdue amounts. The office admin records bank counter payments against an instalment; a payment over the remaining due or of zero is refused. Fee reminders go to students with dues. Receipts for a date range download as Tally receipt vouchers in XML **[A: format based on Tally ERP 9 import samples, not confirmed against TallyPrime]**. Each voucher's narration reads the roll number, student name, instalment label and payment reference. When the instalment has a scholarship, the narration also adds `scholarship <amount>`, and the cash entry stays at the amount paid. Scholarships reduce instalments as set out in section 4.13.

Each payment also has an official receipt on the college profile letterhead, with the amount in words (Indian numbering), a verification hash (HMAC-SHA256 over the receipt's fixed fields, keyed by `BETTER_AUTH_SECRET`) and a QR code. The public `/verify/receipt/<id>?h=<hash>` page shows whether the receipt is valid, with the student's name masked to first name and last initial.

4.4 Student and parent portal (Tier 2)

Students and parents sign in like staff and get the same sidebar layout with their own menu. A student sees today's classes as a carousel that opens on the current or next class, and the week's timetable, attendance per subject with a planner ("attend the next N in a row to reach 75%" or "you can miss N"), a day-by-day register with search, status and date-range filters, marks by component with the final score and a printable mark sheet once moderation locks them, fees with a test-mode Pay button and receipts, with the scholarship awards for each semester and each instalment's scholarship, leave requests, their profile with the admission document checklist, and Termy scoped to their own rows. A parent sees the same for each linked child (switcher in the sidebar), approves the child's leave before the mentor does, reads the college's messages in English or Odia, writes to the mentor, and gives or withdraws DPDP consent for a minor. Every portal query uses the signed-in student, or a child linked to the signed-in parent; no student id is taken from the browser. An alumni account, student or parent, is read-only: the profile and message actions refuse with "Alumni accounts are read-only." (the alumni rule is section 4.14). Mentors decide leave and answer parent notes, and the office answers record corrections, on a Requests page. Approved leave is shown on the register but still counts as absent. Subject faculty (their own subjects) or the office admin decide an appeal once, under Requests > Appeals: accept with the corrected mark or status, or reject with a response. The decision is final. Long lists are paged with `?page=`: register 30 rows, leave 10, messages 15, payments 10.

4.5 Messages and consent

Messages are logged, not sent **[A: no SMS or WhatsApp gateway in scope]**, in English and Odia. Each message is written in the guardian's language: the daily job and the manual fee reminder write one message per distinct guardian language for the student, and a student with no guardian gets English **[A]**. The Messages page shows both language previews of each template, with a note that the Odia text needs review. A student under 18 with no recorded parental consent gets no message; the attempt is logged as blocked, following the DPDP Act's rule for minors. The Odia templates need review by a native speaker before real use.

4.6 Trust dashboard

The trust secretary signs in with the `trust` role and lands on `/trust`, a separate area with its own layout and a guard (`requireTrust()`). The overview shows totals across all campuses: students, the same-day attendance rate, students below the attendance line, fees overdue and open at-risk flags. Campus cards show the same figures side by side, with comparison bars for each measure, and each card opens a read-

only drill-down for that campus. The screens have no write actions. Every campus screen and action also rejects the trust role, so the read-only rule does not depend on the UI. The NAAC returns page (`/trust/naac`) shows each NAAC return (section 4.16) for every campus side by side, read-only, for the current month **[A]**. The same-day attendance rate is session-weighted, and fees overdue means unpaid amounts past their due date **[A]**.

4.7 Term mark sheet and verification

A student or a parent (for the active child) opens the mark sheet from the portal, and an office admin opens it from a student's panel. It lists each subject with its section columns, theory and final score out of 100. Only locked subjects count: a subject that is not yet locked shows as pending, with no numbers. The sheet carries an HMAC-SHA256 hash over the student's id, roll number, semester and each locked subject's code, internal total and final total, keyed by `BETTER_AUTH_SECRET` , and a QR code that links to the public page `/verify/marksheet/<studentId>` . That page checks the hash with a timing-safe comparison and says whether the sheet is genuine. It shows the roll number, semester and locked totals with the student's name masked. Like receipts, the sheet is printed from the browser to PDF on an A4 layout; no PDF library is used.

4.8 Standard reports

An office admin opens Reports and picks one of four standard reports: strength by programme, year, batch, category and gender, attendance shortage for a month, marks summary per subject, and fees against dues by programme. Exams see the first three and accounts see fees, each by its capability (`reports` or `fee_reports` , section 4.15). The accreditation returns are listed beside them (section 4.16). The report lists are fixed in `src/domain/reports.ts` :

- **Strength:** programme, year, batch, category, gender, students.
- **Attendance shortage:** roll no, name, programme, subject, periods held, periods attended, percent.
- **Marks:** subject code, subject, status, students, average internal, average final, min final, max final.
- **Fees:** programme, students, billed, collected, due, overdue (INR).

Month and programme filters narrow the rows, a 200-row preview is shown, and Export CSV downloads the full report with a header row of the labels above **[A]**. Category and gender are read from each student's record, so the strength rows split each programme and year by them **[A]**. The export is audited. Once a day the cron job writes a snapshot of each campus's last full month's standard reports (strength, shortage, marks and fees) into `report_snapshot` . The AICTE and NAAC returns are not snapshotted. The write uses a unique index and does nothing if the month is already stored, so a second run is harmless **[A]**.

4.9 Precomputed results

When office staff lock a subject, the server writes one `result` row per student in the batch, in the same transaction as the lock and its audit row. Missing marks count as zero. Reopening deletes those rows. An accepted mark appeal on a locked subject recomputes the rows for that subject. The portal reads only these rows for locked subjects: the student's marks page and `/api/results` , which returns the signed-in student's results with `Cache-Control: private, max-age=60` . Nothing on the portal recomputes a final score for a locked subject, so results day reads a single row per student, not every mark.

4.10 Applicants and document checklist

Office admins open **Applicants** and see the campus's applicants, with a checklist count per row and an Incomplete or Complete filter. Applicants arrive by CSV import from the same page. The file has the columns `name` , `birth_date` , `category` , `guardian_name` , `guardian_phone` , `guardian_email` and `gender` , and a template downloads from the dialog. The import is all-or-nothing: if any line is wrong, nothing is saved and the dialog lists each bad line with its reason. A file holds at most 500 rows. Categories are General, OBC, SC, ST and EWS **[A]**.

Each programme lists its required documents in **College setup** > **Documents**. A document can carry a category condition (for example, a caste certificate only for SC and ST), and with none it applies to everyone. An applicant's checklist is the set of documents that apply to their category. Each item is pending, received or verified, and the office sets it from the applicant's panel. Every change is written to the audit log. The checklist is complete only when every applicable document is verified and at least one applies. The office can attach a PDF, JPEG or PNG of up to 2 MB to an item; the server checks the size and that the file's content matches its type, by magic bytes (PDF, JPEG or PNG), not the browser's MIME type. It then stores the file in the database (demo scale) **[A]**, and audits each upload or replacement. A file opens only for staff of the applicant's campus with admissions access, or for the student it belongs to on the portal.

4.11 Confirm admission

An office admin confirms an applicant from the applicant's panel. The confirm is blocked, and the screen says why, until every applicable document is verified and, for an applicant under 18 on the confirm date, a parent consent date is recorded **[A]**. A rejected applicant needs a reason, which is stored and audited, and a rejected applicant cannot be confirmed later.

A confirm runs in one transaction:

1. Claim the applicant with a conditional update from `applied` to `confirmed`. A second confirm, or a confirm racing a reject, finds nothing to claim and is refused.
2. Read the checklist again inside the transaction and apply the same checks.
3. Lock the batch row. The roll number is the batch's prefix plus the highest 4-digit suffix in the batch, plus one, padded to four digits. Gaps left by deleted students are not refilled **[A]**. In an empty batch the prefix is the campus code, the admission year and the programme code, for example `ENG-26-CSE-`.
4. Create the student with the roll number, name, batch, birth date, guardian name, guardian phone, parent consent date, category and gender, and copy the checklist into the student's document rows.
5. Create two fee instalments, labelled "Instalment 1" and "Instalment 2", due 30 and 120 days after confirmation. Each is half the semester fee for the student's category (section 4.13), rounded **[A]**.
6. Create the guardian's login: a user, a credential account, the guardian row and the link to the student with relation "Guardian". The email is the applicant's guardian email when it is free, otherwise `firstname.lastname@termwise.demo`, with a number added when that is taken. The password is the demo password **[A]**.
7. Link the applicant to the student (`applicant.student_id`) and write one audit row.

Two concurrent confirms in one batch get consecutive roll numbers, and a repeat confirm of the same applicant is refused. `scripts/confirm-race.mts` checks both against a live database.

4.12 Staff leave

Faculty open **Leave** to apply for casual, medical or on-duty leave, giving from and to dates and a reason. Leave days are inclusive calendar days, so weekends and holidays count **[A]**. Balances are per calendar year: casual 12 days, medical 10 days and on duty unlimited **[A]**. The form refuses a leave that ends before it starts, one that spans a year end (it must be split) **[A]**, one that overlaps the person's own pending or approved leave, and one longer than the balance left for that kind.

Faculty see their balances and their history, and can withdraw a request while it is pending. An office admin or an HR officer of the campus (both hold the `staff_leave` capability) sees the pending requests from that campus's staff, approves or rejects each one, and adds a note that the requester sees. A rejection needs a note. A head of department decides the pending leave of faculty in their own department, and no one else's (section 4.15). The office admin and HR keep the whole campus queue, including faculty leave, as a fallback for departments without a head **[A]**. Nobody decides their own leave, including an office admin or HR. Approved leave is marked "On leave" on that day of the Classes week.

4.13 Fee structure and scholarships

- **Fee structure.** In College setup, the office sets a semester fee per programme and category (`saveFeeStructure`, which needs the `fees` capability). A default option sets the programme's default semester fee. A category with no row uses that default **[A]**. Saving a fee never changes existing instalments; it applies to the instalments created after it **[A]**.
- **Schemes.** A scholarship scheme has a name, a funding source (`government`, `institution` or `non_government`), a kind (a fixed amount in paise, or a percent from 1 to 100), an optional list of categories (empty means all) and an active flag. `saveScheme` needs the `fees` capability.
- **Award.** On the fees page, an office admin with the `fees` capability awards a scheme to a student for one semester. The award is refused when the scheme is inactive, when the student's category is not on the scheme's list, when the scheme is already awarded for that semester (`This scheme is already awarded for that semester.`), or when scholarships already cover the semester's fees (`Scholarships already cover this semester's fees.`) **[A]**. The amount is a percent of the semester's total, rounded to the paisa, or a fixed amount capped at that total. It is spread over the semester's instalments, earliest due first, each capped at what is left unfunded on it. The award is audited as `fees.scholarship.award`.
- **Withdraw.** Withdrawing an award reverses its spread on the instalments, latest due first, and never takes an instalment's scholarship below zero. It is refused with `Already paid; the award can't be withdrawn.` when nothing is due on the semester any more **[A]**. The withdrawal is audited as `fees.scholarship.withdraw`.
- **Dues.** A scholarship reduces an instalment's due, so due = instalment amount − scholarship − payments, floored at zero (section 3).

4.14 Promote and graduate

From College setup, a batch moves to its next semester, or graduates. Both are done by `advanceBatch`, which needs the `setup` capability, and both are audited (`setup.batch.promote` or `setup.batch.graduate`).

1. **Blocker.** The move is refused until every subject of the batch's current semester is locked, and the message lists the codes still open: `Lock every subject first: <codes>.`
2. **Next term.** A batch whose semester is at or past twice the programme's duration in years graduates. Every programme lasts 4 years except the MBA, which lasts 2 **[A]**. Otherwise the next semester is the current one plus one, and the year is the next semester divided by two, rounded up.
3. **Promote.** The finished semester's subjects are stamped with that semester, then cloned as drafts for the new one. Each current student gets two instalments: `Sem N instalment 1/2`, due 30 days later, and `Sem N instalment 2/2`, due 120 days later. Together they make up the student's semester fee for their category: the first is half of it, rounded, and the second is the rest **[A]**. Both start with no scholarship **[A]**.
4. **Graduate.** Each current student in the batch gets `graduated_on` set to today's IST date and becomes alumni. Alumni keep their records, and they are left out of the current-student lists, the risk checks, the Home counts, the daily job and the scholarship counts in the reports. Their portal accounts are read-only (section 4.4).

4.15 Campuses and staff roles

- **Campus switcher.** A user with rows on several campuses picks the active one in the sidebar. The choice is kept in the `termwise_campus` cookie, which is httpOnly, sameSite lax and kept for one year. `getViewer()` uses the row on that campus. A missing, stale or forged value falls back to the user's lowest-id row, so the cookie never gives access to a campus the user does not hold.
- **Capabilities.** Each staff role holds the capabilities below, as listed in `CAPS` in `src/domain/roles.ts`. A screen or action checks the capability it needs, and a role without it is sent to Home.

Role	Capabilities
`faculty`	`teach`
`office_admin`	`campus_students`, `attendance`, `admissions`, `moderate_marks`, `fees`, `risk`, `messages`, `office_requests`, `appeals`, `staff_leave`, `reports`, `fee_reports`, `setup`, `setup_faculty`, `audit`
`admissions`	`admissions`, `campus_students`
`exams`	`moderate_marks`, `reports`, `appeals`
`accounts`	`fees`, `fee_reports`
`hr`	`staff_leave`, `setup_faculty`
`hod`	`dept_students`, `attendance_view`, `marks_view`, `risk_view`, `dept_leave`

- **Audit actor.** An audit row names its actor. For an office role, the name carries the role's label in parentheses, so the log shows which office made the change **[A]**.
- **Seed personas.** The visiting faculty seed persona is faculty at Engineering (Mechanical) and at the Management School, with no department at the Management School. The dual-role persona is faculty at Engineering (Computer Science) and office admin at the Pharmacy College **[A]**.
- **Head of department.** A `hod` sees only their department's students, attendance, marks status and at-risk flags, and decides the pending leave of faculty in that department. A `hod` row with no department sees nothing. A head of department has no fee access and no setup access. The trust role is section 4.6.

4.16 AICTE and NAAC packs

The Reports page lists two accreditation packs beside the standard reports. Each return is one report on the campus. Every return needs the `reports` capability. Each return's note, shown on the page, states the rule it uses.

- **AICTE:** intake (sanctioned seats against everyone admitted in each admission year, alumni included, by category and gender) **[A]**; faculty and HODs by department with designation and qualification; student-faculty ratio against a 1:20 norm, counting students not yet alumni per faculty member of the programme's department **[A]**; cadre ratio of professors, associates and assistants against 1:2:6 per department **[A]**. The sanctioned intake is set per programme and the sanctioned posts per department, both in College setup.
- **NAAC:** 2.1.1 enrolment percentage (admitted in the admission year, alumni included, over sanctioned seats) **[A]**; 2.2 student to full-time teacher ratio, campus-wide, counting faculty and HODs **[A]**; 2.4.1 full-time teachers against sanctioned posts, by department, campus-wide **[A]**; 2.6.3 pass percentage of final-year students **[A]**; 5.1.1 students benefited by scholarships, by funding source **[A]**.
- **Pass rule (2.6.3).** The final year is the batch year equal to the programme's duration, alumni included. A student passes when every locked-subject result is at least the pass mark of 40 out of 100 **[A]**. For a lab subject the pass mark is 40% of its full total. A student with no locked result yet is not counted **[A]**.
- **Norms.** A ratio is below norm when there are more than 20 students per faculty member, or no faculty, for the AICTE student-faculty ratio. For the cadre ratio, it is below norm when professors or associates fall short of 1 and 2 per 6 assistants **[A]**.
- **Pharmacy.** Pharmacy reports to the Pharmacy Council of India (PCI). The AICTE intake return uses the same layout for pharmacy **[A]**.
- **Scope.** The AICTE and NAAC returns are not snapshotted (section 4.8). The NAAC returns page for the trust is in section 4.6.

5. How the AI part works

> **Takeaway: fixed rules decide who is at risk, and the LLM only rewords or plans a query. It cannot write data, and every AI answer shows what it did so staff can check it.**

5.1 At-risk flag

A daily job (and a Run now button) checks every student against the fixed rules: attendance below the cutoff in any subject, attendance dropping by the configured points over 4 weeks, or internal marks below the configured percentage of the maximum of the internal components entered so far. **The rule decides; the LLM never does.** When a student is flagged, the LLM may reword the rule's reason into one plain sentence for the mentor. If the LLM fails, times out or the daily cap is used, the rule's own wording is kept and marked as such. A flag only creates a mentor task; it changes no record and sends nothing to the student. A student with too little data (fewer than the minimum sessions, or no marks entered) is reported as not enough data, not as clear.

5.2 Plain-English questions

Office staff ask questions such as "how many second-year students are below 75% this month".

1. The LLM receives the question and a description of the views the user's role may read, and must return a structured plan: query, clarify or refuse, the SQL, the filters in plain English, and a one-sentence summary. The views each role gets are set in the `ROLE` config in `src/server/assistant/ask.ts`:
   - **office_admin:** all six views, with fees, for the user's campus.
   - **faculty:** five views, without `v_fee_dues`, for the subjects they teach and those batches' students.
   - **student:** their own rows, without `v_risk_flags`, with fees.
   - **admissions:** `v_students` only, no fees.
   - **exams:** `v_students` and `v_marks`, no fees.
   - **accounts:** `v_students` and `v_fee_dues`, for the user's campus.
   - **hod:** `v_students`, `v_subject_attendance`, `v_student_subject_attendance`, `v_marks` and `v_risk_flags`, limited to their department, no fees.
   - **hr:** no views; every question is refused.
2. The SQL is parsed with the Postgres parser (libpg-query) and checked against an allowlist: one SELECT, only the six views, no functions outside a safe list, no writes, no comments or multiple statements.
3. It runs as the `termwise_assistant` role, which can read only those views, inside a read-only transaction with a statement timeout and a row limit. The views filter themselves by the session values set for that transaction: `app.campus_id`, `app.role`, `app.staff_id`, `app.student_id` and `app.department_id`. A head of department's views are limited to `app.department_id`, so a head of department sees only their department. Even a query that passes validation cannot widen that, and an unset role makes the views error rather than show everything.
4. **When it is wrong or unsure:** the answer always shows the filters it used, the row count and the SQL, so staff can see what was counted. An unclear question gets a clarifying question back. A request to change data is refused. Invalid SQL is rejected with a plain message. With no provider or a spent daily cap, the screen says so and points to the normal lists.

5.3 Models and fallback

The chat assistant (Termy) uses these models, in this order. The picker lists only models whose provider key is set, and the first listed is the default. If a call fails, the remaining keyed models are tried in the same order.

Order	Model
1	GPT-OSS 120B (Groq; default when `GROQ_API_KEY` is set)
2	Gemini 3.6 Flash
3	GPT-OSS 20B (Groq)
4	Gemini 3.5 Flash
5	Gemini 3.5 Flash Lite
6	Gemma 4 26B

The at-risk reason wording job uses one model. It is the preferred provider set by `LLM_PROVIDER`: Google (default) gives Gemini 3.6 Flash, and Groq gives GPT-OSS 120B. If that provider's key is missing, the other keyed provider is used. With no keys, the rule's own wording is kept and marked as such. All LLM calls count against the daily cap `LLM_DAILY_CAP` (default 200).

6. UI and design decisions

> **Takeaway: one repeated screen pattern, a phone-first attendance sheet and a restrained palette where colour means status.**

The brief asks for calm, minimal screens used all day, one layout pattern across modules and attendance in under 30 seconds per class.

- **One layout.** Every admin screen is a page header, a short row of counts, and one table with search and filters; selecting a row opens a side panel rather than a new page. Faculty get a bottom tab bar on the phone with four items.
- **Sign-in persona picker.** The demo logins are one searchable list, grouped by campus and role and searchable by name, role or campus. The list is `PERSONAS` in `src/app/(auth)/sign-in/personas.ts`, and `deployment.md` copies it.
- **Campus switcher.** The sidebar shows a campus select only when the login holds rows on two or more campuses. Picking one changes the active campus for every screen (section 4.15).
- **Narrow office roles.** Admissions, exams, accounts and HR get a plain Home with links to their own modules, not the campus dashboard (section 4.15).
- **Restrained palette.** One accent (a petrol blue-green) on neutral greys, in light and dark. Colour carries meaning only for status: present, absent, late, shortage and overdue.
- **Type.** Figtree for text, IBM Plex Mono for roll numbers, percentages and money, so figures line up in tables.
- **Mark attendance.** Built for one thumb: large rows, everyone present by default, a running present and absent count on the Submit button, and a confirm sheet that lists the absent and late names before saving. A class of 60 with five absentees takes five taps, Submit and Confirm.
- **States.** Every list has a loading skeleton, an empty state that says what to do next, and an error state with retry. Offline screens say what is queued and what needs a connection.
- **Key screens:** faculty Home (today's classes and queued sheets), Mark attendance, Marks sheet; admin Home (campus counts and to-dos), Attendance (sessions and conflict queue), Marks (subjects by status, lock and reopen), Fees (dues, payments, reminders, scholarship awards per semester, Tally export), Risk (flags and mentor tasks), Requests (leave, parent notes and appeals), Termy (the assistant, for each role that has views, section 5.2), College setup (subjects with their marks schemes, college profile, required documents, programmes with sanctioned intake, departments with sanctioned posts, faculty with designation and qualification, fee structure, scholarship schemes), Audit log.

- **Batches and promotion.** College setup > Batches has an Advance button on each batch. Its dialog previews the next move (the next semester, or graduation for the final semester), lists any blocking subjects and runs the move on confirm (section 4.14).
- **Students.** The student list has an Alumni or Not alumni filter, so graduated students can be listed on their own (section 4.14). The admissions and leave screens follow the same layout: Applicants (list with the import dialog, and a panel with the checklist and the confirm or reject footer) and Leave (faculty: apply form, balances and history; office admin: pending queue and recent decisions).
- **Phone first on new screens.** Every new screen works at 375 px wide with no horizontal page scroll. Lists switch to a card row on phones, form grids stack to one column, and button rows wrap. Only the table itself scrolls sideways, inside its own box.

7. Testing and security

> **Takeaway: rules are covered by unit tests, whole flows by phone-viewport end-to-end tests, and access is enforced on the server, not hidden in the UI.**

- **Unit tests (Vitest)** cover attendance percentages with labs and late, sync decisions and clock correction, marks validation and the lock, dues and payment validation, risk rules, the assistant SQL validator against injection attempts and per-role view lists, the assistant views' role scoping against a seeded database, Tally XML, the attendance planner, the marks scheme and final score, appeal rules, and amount in words with the receipt hash. Newer unit tests cover the capability map (`CAPS`), head of department scope, the active campus cookie, the fee structure and scholarship spread, promotion and the accreditation rules.
- **End-to-end tests (Playwright, phone viewport)** cover offline marking then sync, the conflict queue, the shared-phone rejection, the marks lock and audited reopen, campus and role limits, fee payments and reminders, the at-risk run, the assistant with no provider, College setup edits, attendance correction, and the student and parent portal (role pages, child switch, leave from student to parent to mentor, appeals raised and decided). Newer specs cover the trust area (overview, drill-down, staff roles kept out), results on lock and reopen, reports (switching, CSV headers, refused requests), Odia messages (templates and a guardian's inbox) and the mark sheet (student, parent, verify page and a tampered hash). Later specs cover access by role and campus, office roles and their modules, the campus switcher, the head of department's department, batch promotion and graduation (with alumni read-only), and fee structure and scholarships.
- **Admissions and leave.** Unit tests in `src/domain/applicants.test.ts` , `admission.test.ts` and `staff-leave.test.ts` cover the checklist and its completion rule, the CSV parser and its 500-row cap, roll number generation, the confirm blockers (documents, minors, status), and the leave balances, overlap and year-end rules. End-to-end specs `e2e/applicants.spec.ts` , `admission.spec.ts` and `staff-leave.spec.ts` cover the checklist and import, the confirm and reject flow, and the leave flow, each with a 375 px check that the page has no horizontal scroll.
- **Unit tests** added for trust totals and ranking, result rows, report aggregation and CSV escaping, the mark sheet hash and message language choice.
- **Run results.** A fresh database run on 2026-11-06 gave 232 unit tests (Vitest, 28 files) and 88 end-to-end runs (Playwright): 69 on a phone viewport and 19 on desktop for sign-in and role access, one desktop run skipped because it changes seed data. The race script passed: two concurrent confirms got consecutive roll numbers, and a repeat confirm was refused. The production build and seed both ran from empty and again on a seeded database.
- **Load test [A].** `scripts/load-results.mts` simulates 7,800 students reading results at once against a running server, with a concurrency of 200. One student session is signed in and reused for every request, so the run measures the route and the database, not sign-in. Run it with

`BASE=http://localhost:3000 EMAIL=ankit.panda@termwise.demo N=7800 C=200 pnpm tsx scripts/load-results.mts`
after starting the server with `E2E=1 BETTER_AUTH_URL=<same origin>`. Measured on a local `pnpm start` build on a 16-core laptop:

```
requests: 7800, concurrency: 200, errors: 0
total time: 6.81 s, req/s: 1146.2
latency ms p50: 169.4, p95: 212.8, p99: 318.3
```

These numbers come from a laptop running the server and the database together, not from the Render free tier, so they show the route's headroom, not production capacity.

- **CI** runs all of it on every push against a fresh Postgres.
- **Security:** no secrets in the repository (only local-only defaults); every input validated with zod at the server; role and campus checked on every query and action; audit log on every change; auth rate-limited; the assistant on a read-only role; students and parents sign in and every portal query is pinned to their own (or their linked children's) record; offline data limited to names and roll numbers and cleared on sign-out. The public receipt verify page shows only the receipt number, date, amount and a masked student name. The public mark sheet verify page shows only the roll number, semester, locked totals and a masked name. The trust role is read-only in code, not just in the UI. Applicant and leave actions check role and campus on the server, and the server refuses a leave decision on one's own request. Access follows the capability map in `CAPS`: each screen and action checks the capability it needs, so a role without it is refused on the server, not only hidden. A head of department's reads and leave decisions are limited to their department. An alumni account is read-only on the server, and its writes are refused with "Alumni accounts are read-only.". The active campus can only select one of the login's own staff rows, so the campus cookie never widens access.

8. What was cut, and what comes next

> **Takeaway: external integrations (SMS, payments, real PDFs) were cut; storage and results-day scale come next.**

Cut from this phase: real SMS or WhatsApp, a real payment gateway, electives and class sections, timetable generation, and real PDF files (the mark sheet and receipts use the browser's print to PDF). Two smaller limits: the mark sheet hash changes if `BETTER_AUTH_SECRET` changes, and the load test reuses one session for every simulated student.

Next, in order:

1. Object storage for admission documents. Today files are stored in the database.
2. Leave across a year end, so a leave can be split and each year's balance is counted correctly.
3. Results day at scale: results are precomputed, but the portal still serves each request from the app. Put a CDN in front of the per-student results page, so the database sees one write per student rather than thousands of reads.
4. University report formats, and saved queries over the same views the assistant uses. AICTE and NAAC are fixed packs in Reports (section 4.16).
5. Rollout: one programme per campus first, then each campus, training one teacher per department as the first line of support.

9. Assumptions

> **Takeaway: 48 choices the brief leaves open, each flagged [A] where it is used.**

Each is a choice the brief does not settle.

1. Late counts as attended for the percentage.
2. A lab is one session covering consecutive periods, weighted by periods.
3. Internal marks out of 40 are faculty-defined components; the demo uses Internal test 20, Assignment 10, Quiz 5 and Attendance 5. Theory is internal 40, mid-term exam 20 and end-term exam 40 (100 in all). A lab scales that theory to 70 and adds lab exam 15, viva 10 and continuous evaluation 5 (see ADR 0001).
4. University upload columns: `RollNo, Name, SubjectCode`, one column per internal component, `InternalTotal`.
5. Risk thresholds: below the cutoff in any subject, a 10-point drop over 4 weeks, internal marks under 50% of the maximum of the internal components entered, at least 10 sessions before judging.
6. Fees in two instalments per year; scholarships reduce an instalment.
7. Tally receipt voucher XML follows ERP 9 import samples.
8. Messages are logged, not sent; parental consent is a recorded date on the student.
9. The demo timetable includes Sunday so a class is always available to mark on the demo day.
10. Faculty see only the subjects they teach and the students they mentor.
11. The sample data is synthetic; the brief's three students appear with their exact figures.
12. Only students (not parents) can appeal; one open appeal per item, with a reason of 10 to 500 characters; a decision is final.
13. Receipt hashes are keyed by `BETTER_AUTH_SECRET`; rotating that secret makes issued receipts fail verification.
14. Report columns are the lists in `src/domain/reports.ts`, copied in section 4.8. The strength report splits by the student's category and gender from the student record.
15. The marks report counts every student in the subject's batch, but its averages, minimum and maximum use only students with complete marks.
16. Monthly report snapshots cover the previous calendar month in IST. The daily job writes them, and a second write for the same month is ignored.
17. The trust attendance rate is session-weighted: sessions attended divided by sessions held, summed across a campus's students, not an average of student percentages.
18. Trust "fees overdue" is the unpaid amount on instalments past their due date.
19. A result's theory total is the internal, mid-term and end-term marks added up raw, out of 100. It is stored in `result.internal`.
20. Each message is written once per distinct guardian language. A student with no guardian gets English. Existing message rows are not rewritten when a guardian's language changes.
21. The load test simulates 7,800 students with one signed-in session reused for every request, so it measures the results route and the database, not sign-in or many sessions.
22. Mark sheet hashes use the same secret as receipts, so rotating `BETTER_AUTH_SECRET` also makes issued mark sheets fail verification.
23. Leave balances are per calendar year: casual 12 days, medical 10 days, on duty unlimited.
24. Leave days are inclusive calendar days. Weekends and holidays count.
25. Leave cannot span a year end. It must be split into two requests.
26. A roll number is the batch's highest roll plus one. Gaps left by deleted students are not refilled.
27. Admission fees are two instalments of half the student's semester fee (the fee for their category, section 4.13), due 30 and 120 days after confirmation.

28. A guardian's login email is the applicant's guardian email when it is free, otherwise `firstname.lastname@termwise.demo`, with the demo password.
29. A minor is under 18 on the confirm date (IST), and needs a parent consent date before confirmation.
30. Applicant categories are General, OBC, SC, ST and EWS.
31. Admission document files are stored in the database as base64 (demo scale), up to 2 MB each, PDF, JPEG or PNG only.
32. Each login holds one staff row per campus. The `staff` table has no unique index, and the app assumes one row per campus (section 3).
33. Saving a fee structure does not change existing instalments; it applies to instalments created after it (section 4.13).
34. A category with no fee structure row uses the programme's default semester fee (section 4.13).
35. A scholarship award is refused when the scholarships already cover the semester's fees ("Scholarships already cover this semester's fees.") (section 4.13).
36. A scholarship withdrawal is refused when nothing is due on that semester any more (section 4.13).
37. The trust NAAC page shows the current month (section 4.6).
38. The strength report splits rows by category and gender from the student record (section 4.8).
39. Promotion splits the semester fee into two instalments: the first is half, rounded, and the second is the rest. Both start with no scholarship (section 4.14).
40. The audit actor name carries the office role's label in parentheses for office roles (section 4.15).
41. AICTE intake counts everyone admitted in each admission year, alumni included, against sanctioned seats. The student-faculty ratio counts students not yet alumni per faculty member of the programme's department. The cadre ratio counts professors, associate and assistant professors per department (section 4.16).
42. NAAC 2.1.1 counts students admitted in the admission year, alumni included, over sanctioned seats. 2.2 is the campus-wide student to full-time teacher ratio. 2.4.1 counts full-time teachers against sanctioned posts by department. 2.6.3 uses the final year, alumni included. 5.1.1 counts students benefited by scholarships by funding source (section 4.16).
43. The NAAC 2.6.3 pass mark is 40% of each subject's full total, so 40 out of 100 for a subject with a full total of 100. A student passes when every locked-subject result reaches it. A student with no locked result yet is not counted (section 4.16).
44. The norms are 1:20 students per faculty member for the AICTE student-faculty ratio, and 1:2:6 for the cadre ratio of professors, associates and assistants (section 4.16).
45. Every programme lasts 4 years, the column default. The MBA is set to 2 years by the alumni seed (section 4.14).
46. Pharmacy reports to the Pharmacy Council of India (PCI). The AICTE intake return uses the same layout for pharmacy (section 4.16).
47. The visiting faculty seed persona is faculty at Engineering (Mechanical) and at the Management School, with no department at the Management School. The dual-role seed persona is faculty at Engineering (Computer Science) and office admin at the Pharmacy College.
48. The office admin and HR keep deciding faculty leave campus-wide, alongside the head of department, so a department without a head still has someone to decide (section 4.12).